

Федеральное государственное бюджетное образовательное учреждение высшего образования «Херсонский государственный педагогический университет»
Институт информационных технологий
Направление подготовки: «09.04.04 Программная инженерия»
Специализация «Разработка программно-информационных систем»
Форма обучения: очная
Курс: 2, Магистратура, группа 12-25РПм
Херсонская область - 2026

Дисциплина:
«Управление конфигурацией и сопровождение программного обеспечения»

Выполнил:
Дмитрий Сергеевич Трифонов
обучающийся 2-го курса

Проверил:
Гуцол Д.В.

ПРАКТИЧЕСКАЯ РАБОТА. Практикум по основам Git в Windows

«Управление конфигурацией и сопровождение программного обеспечения»

I. ЦЕЛЬ И ЗАДАЧИ

1.1. Цель

Целью данной работы является освоение основных возможностей системы контроля версий Git в операционной системе Windows, формирование практических навыков создания и сопровождения локального репозитория, управления изменениями, ветками, коммитами, метками версий и сохранения истории проекта.

1.2. Задачи

- Установить и проверить работу Git for Windows.
- Научиться открывать Git Bash в необходимой рабочей папке.
- Создать локальный Git-репозиторий с основной веткой main.
- Настроить имя и адрес автора коммитов.
- Освоить добавление файлов в индекс с помощью команды git add.
- Создать и проверить историю коммитов с помощью git commit и git log.
- Научиться создавать и переключать ветки Git.
- Объединить ветку feature-plan с веткой main.
- Создать метку версии v0.1.0.
- Сформировать Git bundle, текстовую историю и итоговый ZIP-архив для сдачи работы.

II. ТЕОРЕТИЧЕСКИЕ ОСНОВЫ

2.1. Система контроля версий Git

Git представляет собой распределённую систему контроля версий, предназначенную для отслеживания изменений в программных проектах и хранения их истории. В рамках практикума Git используется локально, поэтому для выполнения основных упражнений не требуется регистрация на внешнем хостинге репозитория.

2.2. Репозиторий, рабочая область и коммиты

Локальный репозиторий Git создаётся командой git init и содержит специальную папку .git с историей и служебными данными. Изменения сначала выполняются в рабочих файлах, затем выбранные файлы добавляются в индекс командой git add, после чего фиксируются в истории командой git commit. Команда git status позволяет контролировать текущее состояние рабочей области.

2.3. Ветки и объединение изменений

Ветка Git представляет отдельную линию разработки. В практикуме основная ветка называется main, а для подготовки плана создаётся ветка feature-plan. После фиксации изменений в feature-plan выполняется переход обратно в main и объединение веток с помощью git merge --ff-only. При таком сценарии история сохраняется без отдельного merge-коммита.

III. МЕТОДЫ И ПОДХОДЫ К ВЫПОЛНЕНИЮ РАБОТЫ

3.1. Подготовка рабочего окружения

Работа выполняется в Windows 10/11 с использованием Git Bash. Сначала проверяется наличие Git командой `git --version`, после чего стартовый ZIP распаковывается в обычную папку. Для практической части используется именно каталог `git-practice`, содержащий исходный `README.txt`.

3.2. Создание репозитория и первой версии проекта

- Создание репозитория выполняется командой `git init -b main`.
- Настраиваются локальные параметры автора с помощью `git config user.name` и `git config user.email`.
- В `README.txt` указывается имя студента.
- Файл добавляется в индекс командой `git add README.txt`.
- Первое состояние проекта фиксируется коммитом `Add project description`.

3.3. Изменение проекта и работа с историей

- В `README.txt` изменяется значение `Status` с `draft` на `in progress`.
- Добавляется собственная строка `Note`.
- Команда `git diff` используется для просмотра изменений до их фиксации.
- Изменения добавляются в индекс и сохраняются коммитом `Update project status`.
- Команда `git log --oneline` позволяет проверить наличие двух коммитов.
- Команда `git status` используется для подтверждения чистого состояния рабочей папки.

3.4. Создание ветки и объединение изменений

После завершения основной части создаётся ветка `feature-plan` командой `git switch -c feature-plan`. В ней создаётся `PLAN.txt` с двумя строками плана работы, после чего файл фиксируется отдельным коммитом `Add work plan`. Затем выполняется переход в `main` и объединение `feature-plan` с помощью `git merge --ff-only`.

3.5. Версионирование и подготовка результата

После объединения создаётся метка учебной версии `v0.1.0` командой `git tag`. Для передачи истории используется `git bundle create`, а его корректность проверяется командой `git bundle verify`. Дополнительно формируется `history.txt` с историей всех веток и меток. Заполненный `report.txt` и файл `work.bundle` вместе с `history.txt` помещаются в каталог `submission` и упаковываются в ZIP-архив.

IV. ПРАКТИЧЕСКОЕ ВЫПОЛНЕНИЕ И АНАЛИТИЧЕСКОЕ ОСМЫСЛЕНИЕ

- В ходе работы был подготовлен локальный репозиторий Git в каталоге git-practice.
- Ветка main использовалась как основная линия разработки.
- README.txt был сохранён в истории двумя последовательными коммитами, отражающими исходное и изменённое состояние проекта.
- Ветка feature-plan позволила выполнить отдельную разработку PLAN.txt без непосредственного изменения main.
- После выполнения git merge --ff-only feature-plan файл PLAN.txt появился в основной ветке.
- Итоговая история содержит три коммита: Add project description, Update project status и Add work plan.
- Метка v0.1.0 указывает на итоговое состояние проекта после объединения изменений.
- Команда git status используется как основной инструмент контроля чистоты рабочего дерева.
- Git bundle позволяет сохранить историю репозитория в переносимом файле без необходимости использования удалённого сервера.
- Дополнительное упражнение с conflict-demo демонстрирует ситуацию конфликта при изменении одной строки в разных ветках и необходимость ручного выбора итогового содержимого.

V. ВЫВОДЫ

В результате выполнения практикума были освоены базовые операции системы контроля версий Git в Windows: создание локального репозитория, настройка автора, отслеживание изменений, добавление файлов в индекс, создание коммитов, просмотр истории, работа с ветками, объединение изменений и создание меток версий. Был сформирован репозиторий с файлами README.txt и PLAN.txt, тремя коммитами, ветками main и feature-plan и меткой v0.1.0. Дополнительно были подготовлены bundle, history.txt и краткий отчёт для последующей упаковки в ZIP-архив. Практическая работа показала, что Git позволяет сохранять последовательную историю изменений, организовывать параллельную разработку и контролировать состояние программного проекта, что является важной частью управления конфигурацией и сопровождения программного обеспечения.